

Step 1

Choose RMW factor

Step 2

I use the 25 size and BE/ME portfolios as the test assets and RMW (Robust Minus Weak) as the additional factor. I expect RMW to help price these portfolios because profitability captures a dimension of expected returns that is not included in the CAPM market factor and may vary systematically across firms with different size and book-to-market characteristics.

Before examining the results, my hypothesis is that adding RMW to the market factor will reduce the GRS F-statistic relative to the CAPM, although I do not expect it to eliminate all pricing errors. I expect the largest alpha reductions to occur for portfolios with stronger RMW exposures, while portfolios with weak profitability exposure should show smaller improvements.

Economically, RMW may earn a positive risk premium because firms with weak profitability could be riskier and therefore require different expected returns, or the profitability premium may reflect systematic mispricing if investors do not fully incorporate information about firms' profitability into prices. The test will examine whether adding RMW materially reduces the pricing errors of the 25 portfolios beyond what market beta alone can explain.

Question 3 a

```
In [ ]: import pandas as pd
import numpy as np
import statsmodels.api as sm
from scipy.stats import f

excel_file = "Problem_Set4_2026.xlsx"
factor_file = "F-F_Research_Data_5_Factors_2x3.csv"
pd.set_option("display.max_columns", 35)
pd.set_option("display.width", 200)

book = pd.ExcelFile(excel_file)
print("Excel sheets:", book.sheet_names)

portfolio_raw = pd.read_excel(
    book, sheet_name="25 size and BEME portfolios", header=None
)
market_raw = pd.read_excel(
    book, sheet_name="Market, Rf", header=None
)
```

```

print("\nPortfolio header rows:")
print(portfolio_raw.iloc[:3].to_string(index=False, header=False))

print("\nMarket header rows:")
print(market_raw.iloc[:2].to_string(index=False, header=False))

# Preserve the workbook's original size-first, BE/ME-second order.
portfolio_cols = [
    f"S{s}_BM{b}" for s in range(1, 6) for b in range(1, 6)
]

portfolios = portfolio_raw.iloc[3:].copy()
assert portfolios.shape[1] == 26
portfolios.columns = ["Date"] + portfolio_cols

market = market_raw.iloc[2:].copy()
assert market.shape[1] == 3
assert market_raw.iloc[1, 1:].tolist() == ["RM-RF", "RF"]
market.columns = ["Date", "RM-RF", "RF"]

# The CSV header is on line 5.
ff = pd.read_csv(factor_file, skiprows=4)
ff.columns = ff.columns.str.strip()
ff = ff.rename(columns={ff.columns[0]: "Date"})

print("\nCSV columns:", ff.columns.tolist())

# Monthly dates have six digits. Annual dates have four.
date_text = ff["Date"].astype(str).str.strip()
monthly_rows = date_text.str.fullmatch(r"\d{6}")

print("CSV rows outside monthly section:", int((~monthly_rows).sum()))

# Retain only RMW from the external factor file.
rmw = ff.loc[monthly_rows, ["Date", "RMW"]].copy()

for name, data in [
    ("Portfolios", portfolios),
    ("Workbook market and RF", market),
    ("Monthly RMW", rmw)
]:
    print("\n" + name)
    print("Dimensions:", data.shape)
    print("Columns:", data.columns.tolist())
    print(data.head().to_string(index=False))
    print("Data types:")
    print(data.dtypes)

```

Portfolio header rows:

Market header rows:

CSV columns: ['Date', 'Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA', 'RF']

Portfolios

```
Columns: ['Date', 'S1_BM1', 'S1_BM2', 'S1_BM3', 'S1_BM4', 'S1_BM5', 'S2_BM1', 'S2_BM2', 'S2_BM3', 'S2_BM4', 'S2_BM5', 'S3_BM1', 'S3_BM2', 'S3_BM3', 'S3_BM4', 'S3_BM5', 'S4_BM1', 'S4_BM2', 'S4_BM3', 'S4_BM4', 'S4_BM5', 'S5_BM1', 'S5_BM2', 'S5_BM3', 'S5_BM4', 'S5_BM5']
```

Data types:

3/30

```

S3_BM3    float64
S3_BM4    float64
S3_BM5     object
S4_BM1     object
S4_BM2    float64
S4_BM3    float64
S4_BM4    float64
S4_BM5     object
S5_BM1     object
S5_BM2     object
S5_BM3     object
S5_BM4     object
S5_BM5     object
dtype: object

```

```

Workbook market and RF
Dimensions: (1200, 3)
Columns: ['Date', 'RM-RF', 'RF']
   Date  RM-RF  RF
192607.0  2.89  0.22
192608.0  2.64  0.25
192609.0  0.38  0.23
192610.0 -3.27  0.32
192611.0  2.54  0.31
Data types:
Date      float64
RM-RF     object
RF        object
dtype: object

```

```

Monthly RMW
Dimensions: (758, 2)
Columns: ['Date', 'RMW']
   Date  RMW
196307  0.64
196308  0.40
196309 -0.78
196310  2.79
196311 -0.43
Data types:
Date      str
RMW       str
dtype: object

```

```

In [2]: sources = [
        ("Portfolios", portfolios),
        ("Workbook market and RF", market),
        ("Monthly RMW", rmw)
    ]

    for name, data in sources:
        dates = pd.to_numeric(data["Date"], errors="raise")

        assert dates.notna().all(), name + ": missing dates"
        assert (dates % 1 == 0).all(), name + ": invalid YYYYMM dates"

```

```

data["Date"] = pd.to_datetime(
    dates.astype(int).astype(str), format="%Y%m", errors="raise"
).dt.to_period("M")

value_cols = data.columns.drop("Date")
data[value_cols] = data[value_cols].apply(
    pd.to_numeric, errors="raise"
)

# Recognize standard missing-return codes explicitly.
sentinel_count = data[value_cols].isin([-99.99, -999]).sum().sum()
print(f"\n{name}: missing-value codes = {sentinel_count}")

data[value_cols] = data[value_cols].replace(
    [-99.99, -999], np.nan
)

print("Missing values before merging:")
print(data[value_cols].isna().sum().to_string())
print("Rows with any missing return:",
      int(data[value_cols].isna().any(axis=1).sum()))
print("Duplicate dates:", int(data["Date"].duplicated().sum()))
print("Range:", data["Date"].min(), "to", data["Date"].max())

assert not data["Date"].duplicated().any(), name + ": duplicate dates"

# Both models will use this single merged sample.
common = portfolios.merge(
    market, on="Date", how="inner", validate="one_to_one"
)
common = common.merge(
    rmw, on="Date", how="inner", validate="one_to_one"
)
common = common.sort_values("Date").set_index("Date")

for name, data in sources:
    excluded = data.loc[~data["Date"].isin(common.index), "Date"]
    print(f"\n{name}: {len(excluded)} months excluded by date overlap")
    if len(excluded) > 0:
        print("Excluded range:", excluded.min(), "to", excluded.max())

print("\nMissing values after merging:")
print(common.isna().sum().to_string())
print("Rows with any missing value:",
      int(common.isna().any(axis=1).sum()))

assert len(common) > 0, "No overlapping observations."
assert not common.isna().any().any(), "Missing values remain. Stop here."
assert np.isfinite(common.to_numpy()).all(), "Nonfinite values remain."

expected_months = pd.period_range(
    common.index.min(), common.index.max(), freq="M"
)
assert common.index.equals(expected_months), "There are gaps in the sample."

print("\nCommon sample:")

```

```
print("Start:", common.index.min())
print("End:", common.index.max())
print("T:", len(common))
print("Units: percentage points for portfolios, RM-RF, RF, and RMW.")
print("\nCleaned data types:")
print(common.dtypes)
```

Portfolios: missing-value codes = 0

Missing values before merging:

S1_BM1	0
S1_BM2	0
S1_BM3	0
S1_BM4	0
S1_BM5	0
S2_BM1	0
S2_BM2	0
S2_BM3	0
S2_BM4	0
S2_BM5	0
S3_BM1	0
S3_BM2	0
S3_BM3	0
S3_BM4	0
S3_BM5	0
S4_BM1	0
S4_BM2	0
S4_BM3	0
S4_BM4	0
S4_BM5	0
S5_BM1	0
S5_BM2	0
S5_BM3	0
S5_BM4	0
S5_BM5	0

Rows with any missing return: 0

Duplicate dates: 0

Range: 1926-07 to 2026-06

Workbook market and RF: missing-value codes = 0

Missing values before merging:

RM-RF	0
RF	0

Rows with any missing return: 0

Duplicate dates: 0

Range: 1926-07 to 2026-06

Monthly RMW: missing-value codes = 0

Missing values before merging:

RMW	0
-----	---

Rows with any missing return: 0

Duplicate dates: 0

Range: 1963-07 to 2026-08

Portfolios: 444 months excluded by date overlap

Excluded range: 1926-07 to 1963-06

Workbook market and RF: 444 months excluded by date overlap

Excluded range: 1926-07 to 1963-06

Monthly RMW: 2 months excluded by date overlap

Excluded range: 2026-07 to 2026-08

Missing values after merging:

S1_BM1	0
S1_BM2	0
S1_BM3	0
S1_BM4	0
S1_BM5	0
S2_BM1	0
S2_BM2	0
S2_BM3	0
S2_BM4	0
S2_BM5	0
S3_BM1	0
S3_BM2	0
S3_BM3	0
S3_BM4	0
S3_BM5	0
S4_BM1	0
S4_BM2	0
S4_BM3	0
S4_BM4	0
S4_BM5	0
S5_BM1	0
S5_BM2	0
S5_BM3	0
S5_BM4	0
S5_BM5	0
RM-RF	0
RF	0
RMW	0

Rows with any missing value: 0

Common sample:

Start: 1963-07

End: 2026-06

T: 756

Units: percentage points for portfolios, RM-RF, RF, and RMW.

Cleaned data types:

S1_BM1	float64
S1_BM2	float64
S1_BM3	float64
S1_BM4	float64
S1_BM5	float64
S2_BM1	float64
S2_BM2	float64
S2_BM3	float64
S2_BM4	float64
S2_BM5	float64
S3_BM1	float64
S3_BM2	float64
S3_BM3	float64
S3_BM4	float64
S3_BM5	float64
S4_BM1	float64
S4_BM2	float64
S4_BM3	float64
S4_BM4	float64

```

S4_BM5    float64
S5_BM1    float64
S5_BM2    float64
S5_BM3    float64
S5_BM4    float64
S5_BM5    float64
RM-RF      float64
RF         float64
RMW        float64
dtype: object

```

```
In [3]: Y = common[portfolio_cols].sub(common["RF"], axis=0)
```

```

F_capm = common[["RM-RF"]].copy()
F_aug = common[["RM-RF", "RMW"]].copy()

assert Y.index.equals(F_capm.index)
assert Y.index.equals(F_aug.index)
assert Y.shape[1] == 25

print("Portfolio excess returns = raw portfolio returns - workbook RF.")
print("RM-RF and RMW are used directly.")
print("All returns remain in percentage points.")
print(Y.head().to_string())

```

Portfolio excess returns = raw portfolio returns - workbook RF.

RM-RF and RMW are used directly.

All returns remain in percentage points.

	S1_BM1	S1_BM2	S1_BM3	S1_BM4	S1_BM5	S2_BM1	S2_BM2	S2_BM3	S2_BM4	S2_BM5	S3_BM1	S3_BM2	S3_BM3	S3_BM4	S3_BM5	S4_BM1	S4_BM2	S4_BM3	S4_BM4	S4_BM5	S5_BM1	S5_BM2	S5_BM3	S5_BM4	S5_BM5
Date																									
1963-07	0.8587	-0.6332	0.4523	-0.3113	-1.5147	-2.0776	-0.0771	-1.2849	-2.2449	-1.4580	-2.6227	-1.5947	-1.0083	-2.4029	-2.0226	-1.1815	-2.0433	-2.1868	-1.8445	-2.1274	-0.1309	0.2139	0.8660	-0.6985	-1.3745
1963-08	3.9896	1.1230	1.2417	2.2568	4.4144	5.3203	4.2720	4.1950	4.2162	7.9951	4.8379	5.0555	5.1331	4.5913	4.9062	5.3254	4.4969	6.0016	7.4441	5.0956	5.5323	4.0133	4.3841	7.9204	6.1484
1963-09	-2.0043	0.3504	-1.2707	-1.7915	-0.6284	-4.3225	-1.7772	-1.1338	-1.7635	-3.1943	-4.6015	-2.0690	-0.6003	-1.2441	-2.1027	-2.9344	-2.4163	-2.0582	-4.2341	-2.2702	-1.6452	-1.0781	-1.1197	-0.4612	-3.7733
1963-10	0.0878	-1.0229	1.0166	-0.0996	2.0811	0.9026	3.9511	2.0626	2.0158	3.6414	2.1051	0.1330	2.4561	2.2339	0.7796	-0.5315	0.4090	2.2314	4.5624	0.3238	5.0361	1.4520	-0.6254	2.1276	0.1802
1963-11	-3.6019	-4.1136	-2.0593	-1.3235	-1.3777	-4.5296	-2.0184	-1.0545	-0.3254	-0.3850	-3.2657	-1.6481	-1.3192	-1.0766	-1.4863	-1.1783	-0.9011	-1.0216	1.0896	3.2707	-1.5369	0.7380	-1.9614	-2.4016	1.0796

```
In [4]: X_capm = sm.add_constant(F_capm, has_constant="add")
```

```

capm_rows = []
capm_residuals = pd.DataFrame(index=Y.index)
capm_fitted = pd.DataFrame(index=Y.index)

for portfolio in portfolio_cols:
    model = sm.OLS(Y[portfolio], X_capm, missing="raise").fit()

```

```

capm_rows.append({
    "Portfolio": portfolio,
    "Alpha": model.params["const"],
    "Alpha SE": model.bse["const"],
    "Alpha t-stat": model.tvalues["const"],
    "Market beta": model.params["RM-RF"],
    "Market beta t-stat": model.tvalues["RM-RF"]
})

capm_residuals[portfolio] = model.resid
capm_fitted[portfolio] = model.fittedvalues

capm_results = pd.DataFrame(capm_rows).set_index("Portfolio")
print(capm_results.to_string(float_format=lambda x: f"{x:.6f}"))

```

Portfolio	Alpha	Alpha SE	Alpha t-stat	Market beta	Market beta t-stat
S1_BM1	-0.521850	0.178183	-2.928735	1.416281	35.776716
S1_BM2	0.059483	0.157362	0.378003	1.218823	34.862349
S1_BM3	0.137430	0.129548	1.060841	1.110712	38.590975
S1_BM4	0.347082	0.131246	2.644521	1.033757	35.452757
S1_BM5	0.473040	0.149931	3.155047	1.064179	31.947645
S2_BM1	-0.265817	0.135783	-1.957654	1.378430	45.693485
S2_BM2	0.102585	0.111802	0.917562	1.182825	47.619870
S2_BM3	0.245106	0.103530	2.367501	1.081168	47.005116
S2_BM4	0.291446	0.106096	2.746997	1.015431	43.079080
S2_BM5	0.319903	0.131640	2.430130	1.140068	38.981468
S3_BM1	-0.219664	0.109178	-2.011970	1.305802	53.834067
S3_BM2	0.124490	0.086792	1.434342	1.115876	57.869709
S3_BM3	0.143888	0.085191	1.689005	1.014145	53.582486
S3_BM4	0.271832	0.095434	2.848367	0.996213	46.985534
S3_BM5	0.343818	0.124506	2.761454	1.083244	39.160810
S4_BM1	-0.069075	0.083649	-0.825770	1.191907	64.135158
S4_BM2	0.015816	0.068141	0.232109	1.074965	71.007533
S4_BM3	0.142890	0.078835	1.812515	1.010487	57.693475
S4_BM4	0.276255	0.090951	3.037397	0.984217	48.707726
S4_BM5	0.221058	0.120072	1.841043	1.076390	40.349933
S5_BM1	0.019774	0.059044	0.334903	0.990808	75.531390
S5_BM2	0.029546	0.057649	0.512516	0.923417	72.098051
S5_BM3	0.091486	0.074574	1.226788	0.859969	51.905424
S5_BM4	0.020895	0.095496	0.218806	0.884442	41.687131
S5_BM5	0.194488	0.132008	1.473302	1.002421	34.179471

```

In [5]: def grs_test(alpha, residuals, factors):
        alpha = np.asarray(alpha, dtype=float).reshape(-1)
        E = np.asarray(residuals, dtype=float)
        F = np.asarray(factors, dtype=float)

        assert E.ndim == 2 and F.ndim == 2

        T, N = E.shape
        K = F.shape[1]

        assert F.shape[0] == T
        assert len(alpha) == N

```

```

assert T > N + K
assert np.isfinite(alpha).all()
assert np.isfinite(E).all()
assert np.isfinite(F).all()

# Covariance of OLS residuals, not raw portfolio returns.
sigma = E.T @ E / (T - K - 1)

factor_mean = F.mean(axis=0)
centered_factors = F - factor_mean
omega = centered_factors.T @ centered_factors / (T - 1)

# Stop if covariance matrices are not positive definite.
np.linalg.cholesky(sigma)
np.linalg.cholesky(omega)

alpha_term = alpha @ np.linalg.solve(sigma, alpha)
factor_term = factor_mean @ np.linalg.solve(omega, factor_mean)

statistic = ((T - N - K) / N) * alpha_term / (1 + factor_term)
p_value = f.sf(statistic, N, T - N - K)

return {
    "T": T,
    "N": N,
    "K": K,
    "GRS F-statistic": float(statistic),
    "p-value": float(p_value),
    "Residual denominator": T - K - 1,
    "Factor denominator": T - 1,
    "F df numerator": N,
    "F df denominator": T - N - K
}

capm_grs = grs_test(
    capm_results["Alpha"], capm_residuais, F_capm
)

print(pd.Series(capm_grs).to_string())

```

```

T                7.560000e+02
N                2.500000e+01
K                1.000000e+00
GRS F-statistic  4.164336e+00
p-value         9.853287e-11
Residual denominator  7.540000e+02
Factor denominator  7.550000e+02
F df numerator    2.500000e+01
F df denominator  7.300000e+02

```

In [6]: `X_aug = sm.add_constant(F_aug, has_constant="add")`

```

aug_rows = []
aug_residuais = pd.DataFrame(index=Y.index)
aug_fitted = pd.DataFrame(index=Y.index)

```

```
for portfolio in portfolio_cols:
    model = sm.OLS(Y[portfolio], X_aug, missing="raise").fit()

    aug_rows.append({
        "Portfolio": portfolio,
        "Alpha": model.params["const"],
        "Alpha SE": model.bse["const"],
        "Alpha t-stat": model.tvalues["const"],
        "Market beta": model.params["RM-RF"],
        "Market beta t-stat": model.tvalues["RM-RF"],
        "RMW beta": model.params["RMW"],
        "RMW beta t-stat": model.tvalues["RMW"]
    })

    aug_residuals[portfolio] = model.resid
    aug_fitted[portfolio] = model.fittedvalues

aug_results = pd.DataFrame(aug_rows).set_index("Portfolio")
print(aug_results.to_string(float_format=lambda x: f"{x:.6f}"))
```

	Alpha	Alpha SE	Alpha t-stat	Market beta	Market beta t-stat
RMW beta	RMW beta	t-stat			
Portfolio					
S1_BM1	-0.211694	0.158966	-1.331696	1.316090	36.896938
-1.029478	-14.548701				
S1_BM2	0.327770	0.141191	2.321462	1.132157	35.736173
-0.890504	-14.169017				
S1_BM3	0.296334	0.123484	2.399775	1.059381	38.233977
-0.527438	-9.595584				
S1_BM4	0.487168	0.126945	3.837622	0.988504	34.703280
-0.464978	-8.228621				
S1_BM5	0.598277	0.147524	4.055460	1.023723	30.926329
-0.415691	-6.330228				
S2_BM1	-0.069699	0.126331	-0.551716	1.315077	46.392563
-0.650960	-11.575887				
S2_BM2	0.213308	0.108813	1.960328	1.147058	46.980146
-0.367516	-7.587664				
S2_BM3	0.284879	0.103983	2.739681	1.068320	45.787779
-0.132014	-2.852144				
S2_BM4	0.343806	0.106186	3.237785	0.998517	41.908154
-0.173793	-3.676875				
S2_BM5	0.405280	0.130888	3.096381	1.112488	37.879413
-0.283385	-4.863928				
S3_BM1	-0.088994	0.104372	-0.852661	1.263591	53.954898
-0.433723	-9.335527				
S3_BM2	0.157596	0.087179	1.807739	1.105182	56.497870
-0.109888	-2.831715				
S3_BM3	0.139722	0.086017	1.624346	1.015491	52.613766
0.013829	0.361176				
S3_BM4	0.275619	0.096363	2.860227	0.994989	46.016991
-0.012571	-0.293062				
S3_BM5	0.358736	0.125659	2.854836	1.078425	38.247594
-0.049517	-0.885256				
S4_BM1	0.020274	0.080903	0.250591	1.163044	64.067656
-0.296569	-8.235143				
S4_BM2	-0.001669	0.068643	-0.024317	1.080613	70.158662
0.058038	1.899427				
S4_BM3	0.108966	0.079071	1.378075	1.021445	57.571263
0.112602	3.199181				
S4_BM4	0.269690	0.091824	2.937032	0.986338	47.871559
0.021792	0.533145				
S4_BM5	0.235950	0.121180	1.947108	1.071579	39.409644
-0.049429	-0.916357				
S5_BM1	-0.049362	0.056585	-0.872359	1.013142	79.795472
0.229480	9.110771				
S5_BM2	-0.031051	0.055836	-0.556116	0.942992	75.267227
0.201136	8.092622				
S5_BM3	0.053745	0.074602	0.720423	0.872161	52.102069
0.125272	3.772381				
S5_BM4	-0.024922	0.095623	-0.260630	0.899242	41.910493
0.152078	3.572848				
S5_BM5	0.234433	0.132857	1.764546	0.989518	33.192978
-0.132586	-2.241941				

```
In [7]: aug_grs = grs_test(
        aug_results["Alpha"], aug_residuals, F_aug
```

```
)

print(pd.Series(aug_grs).to_string())
```

```
T                7.560000e+02
N                2.500000e+01
K                2.000000e+00
GRS F-statistic   3.723390e+00
p-value          4.213838e-09
Residual denominator  7.530000e+02
Factor denominator  7.550000e+02
F df numerator     2.500000e+01
F df denominator    7.290000e+02
```

```
In [8]: validation_passed = False
validation_rows = []
audit_rows = []

models = [
    ("CAPM", capm_results, capm_residuals, capm_fitted,
     F_capm, capm_grs, 1),
    ("Market + RMW", aug_results, aug_residuals, aug_fitted,
     F_aug, aug_grs, 2)
]

for name, results, residuals, fitted, factors, result, expected_k in models:
    alpha = results["Alpha"].to_numpy()
    original_grs = result["GRS F-statistic"]

    # Check 1: zero alpha.
    zero_grs = grs_test(
        np.zeros(len(alpha)), residuals, factors
    )["GRS F-statistic"]

    # Check 2: reverse the portfolio order consistently.
    reorder_grs = grs_test(
        alpha[::-1], residuals.iloc[:, ::-1], factors
    )["GRS F-statistic"]

    # Check 3: scale all returns and factors, then refit.
    scaled_X = sm.add_constant(factors * 100, has_constant="add")
    scaled_alpha = []
    scaled_residuals = pd.DataFrame(index=Y.index)

    for portfolio in portfolio_cols:
        scaled_model = sm.OLS(Y[portfolio] * 100, scaled_X).fit()
        scaled_alpha.append(scaled_model.params["const"])
        scaled_residuals[portfolio] = scaled_model.resid

    scale_grs = grs_test(
        scaled_alpha, scaled_residuals, factors * 100
    )["GRS F-statistic"]

    checks = [
        ("Zero alpha", zero_grs,
         np.isclose(zero_grs, 0, rtol=0, atol=1e-12)),
```

```

        ("Reorder invariance", reorder_grs,
         np.isclose(reorder_grs, original_grs, rtol=1e-9, atol=1e-12)),
        ("Scale invariance", scale_grs,
         np.isclose(scale_grs, original_grs, rtol=1e-9, atol=1e-12))
    ]

    for check, value, passed in checks:
        validation_rows.append({
            "Model": name,
            "Check": check,
            "GRS": value,
            "Result": "PASS" if passed else "FAIL"
        })

    T, N = residuals.shape
    K = factors.shape[1]

    # Verify residuals equal observed excess returns minus fitted returns.
    residual_check = np.allclose(
        residuals.to_numpy(),
        (Y - fitted).to_numpy(),
        rtol=1e-10, atol=1e-10
    )

    # OLS residuals should be orthogonal to the regressors.
    X = sm.add_constant(factors, has_constant="add").to_numpy()
    orthogonal_check = np.allclose(
        X.T @ residuals.to_numpy(), 0, rtol=0, atol=1e-7
    )

    specification_check = (
        K == expected_k
        and result["K"] == expected_k
        and result["N"] == 25
        and result["Residual denominator"] == T - K - 1
        and result["Factor denominator"] == T - 1
        and result["F df numerator"] == N
        and result["F df denominator"] == T - N - K
    )

    audit_rows.append({
        "Model": name,
        "K": K,
        "Residual denominator": result["Residual denominator"],
        "Factor denominator": result["Factor denominator"],
        "F degrees of freedom": (N, T - N - K),
        "OLS residual check": "PASS" if residual_check else "FAIL",
        "Orthogonality": "PASS" if orthogonal_check else "FAIL",
        "Specification": "PASS" if specification_check else "FAIL"
    })

    validation_results = pd.DataFrame(validation_rows)
    grs_audit = pd.DataFrame(audit_rows)

    print(validation_results.to_string(index=False))
    print("\nGRS implementation audit:")

```

```
print(grs_audit.to_string(index=False))

assert validation_results["Result"].eq("PASS").all(), "GRS validation failed
assert grs_audit["OLS residual check"].eq("PASS").all(), "Residual check fai
assert grs_audit["Orthogonality"].eq("PASS").all(), "OLS orthogonality faile
assert grs_audit["Specification"].eq("PASS").all(), "GRS specification faile

validation_passed = True
print("\nAll checks passed. Continue to the comparison tables.")
```

	Model	Check	GRS	Result
	CAPM	Zero alpha	0.000000	PASS
	CAPM Reorder	invariance	4.164336	PASS
	CAPM Scale	invariance	4.164336	PASS
Market + RMW		Zero alpha	0.000000	PASS
Market + RMW Reorder		invariance	3.723390	PASS
Market + RMW Scale		invariance	3.723390	PASS

GRS implementation audit:

	Model	K	Residual denominator	Factor denominator	F	degrees of freed
	om OLS residual check		Orthogonality	Specification		
0)	CAPM	1	754	755		(25, 73)
			PASS	PASS	PASS	
9)	Market + RMW	2	753	755		(25, 72)
			PASS	PASS	PASS	

All checks passed. Continue to the comparison tables.

```
In [ ]: assert validation_passed, "Run the validation cell first."
```

```
comparison = pd.DataFrame([
    {
        "Model": "CAPM",
        "Factors": "RM-RF",
        "T": capm_grs["T"],
        "N": capm_grs["N"],
        "K": capm_grs["K"],
        "GRS F-statistic": capm_grs["GRS F-statistic"],
        "p-value": capm_grs["p-value"],
        "Average RMW return": np.nan
    },
    {
        "Model": "Augmented model",
        "Factors": "RM-RF + RMW",
        "T": aug_grs["T"],
        "N": aug_grs["N"],
        "K": aug_grs["K"],
        "GRS F-statistic": aug_grs["GRS F-statistic"],
        "p-value": aug_grs["p-value"],
        "Average RMW return": common["RMW"].mean()
    }
])

assert capm_grs["GRS F-statistic"] > 0

grs_reduction = (
```

```

    (capm_grs["GRS F-statistic"] - aug_grs["GRS F-statistic"])
    / capm_grs["GRS F-statistic"] * 100
)

print(comparison.to_string(
    index=False, na_rep="N/A",
    formatters={
        "GRS F-statistic": "{:.6f}".format,
        "p-value": "{:.6e}".format,
        "Average RMW return": "{:.6f}".format
    }
))
print(f"\nGRS percentage reduction: {grs_reduction:.6f}%")

```

	Model	Factors	T	N	K	GRS F-statistic	p-value	Average R
MW return								
	CAPM	RM-RF	756	25	1	4.164336	9.853287e-11	
N/A								
Augmented model	RM-RF + RMW		756	25	2	3.723390	4.213838e-09	
0.242804								

GRS percentage reduction: 10.588612%

```

In [ ]: assert validation_passed, "Run the validation cell first."

alpha_comparison = pd.DataFrame({
    "Portfolio": portfolio_cols,
    "CAPM alpha": capm_results["Alpha"].to_numpy(),
    "CAPM alpha t-stat": capm_results["Alpha t-stat"].to_numpy(),
    "Market + RMW alpha": aug_results["Alpha"].to_numpy(),
    "Market + RMW alpha t-stat": aug_results["Alpha t-stat"].to_numpy(),
    "RMW beta": aug_results["RMW beta"].to_numpy(),
    "RMW beta t-stat": aug_results["RMW beta t-stat"].to_numpy()
})

alpha_comparison["Absolute alpha reduction"] = (
    alpha_comparison["CAPM alpha"].abs()
    - alpha_comparison["Market + RMW alpha"].abs()
)

alpha_sorted = alpha_comparison.sort_values(
    "Absolute alpha reduction", ascending=False
).copy()

print("Original portfolio order:")
print(alpha_comparison.to_string(
    index=False, float_format=lambda x: f"{x:.6f}"
))

print("\nSorted by absolute alpha reduction:")
print(alpha_sorted.to_string(
    index=False, float_format=lambda x: f"{x:.6f}"
))

```

Original portfolio order:

Portfolio	CAPM alpha alpha t-stat	CAPM alpha t-stat RMW beta	Market + RMW alpha Absolute alpha reduction	Market + RMW a
S1_BM1	-0.521850	-2.928735	-0.211694	
-1.331696	-1.029478	-14.548701	0.310156	
S1_BM2	0.059483	0.378003	0.327770	
2.321462	-0.890504	-14.169017	-0.268286	
S1_BM3	0.137430	1.060841	0.296334	
2.399775	-0.527438	-9.595584	-0.158904	
S1_BM4	0.347082	2.644521	0.487168	
3.837622	-0.464978	-8.228621	-0.140086	
S1_BM5	0.473040	3.155047	0.598277	
4.055460	-0.415691	-6.330228	-0.125237	
S2_BM1	-0.265817	-1.957654	-0.069699	
-0.551716	-0.650960	-11.575887	0.196118	
S2_BM2	0.102585	0.917562	0.213308	
1.960328	-0.367516	-7.587664	-0.110723	
S2_BM3	0.245106	2.367501	0.284879	
2.739681	-0.132014	-2.852144	-0.039773	
S2_BM4	0.291446	2.746997	0.343806	
3.237785	-0.173793	-3.676875	-0.052360	
S2_BM5	0.319903	2.430130	0.405280	
3.096381	-0.283385	-4.863928	-0.085377	
S3_BM1	-0.219664	-2.011970	-0.088994	
-0.852661	-0.433723	-9.335527	0.130670	
S3_BM2	0.124490	1.434342	0.157596	
1.807739	-0.109888	-2.831715	-0.033106	
S3_BM3	0.143888	1.689005	0.139722	
1.624346	0.013829	0.361176	0.004166	
S3_BM4	0.271832	2.848367	0.275619	
2.860227	-0.012571	-0.293062	-0.003787	
S3_BM5	0.343818	2.761454	0.358736	
2.854836	-0.049517	-0.885256	-0.014918	
S4_BM1	-0.069075	-0.825770	0.020274	
0.250591	-0.296569	-8.235143	0.048802	
S4_BM2	0.015816	0.232109	-0.001669	
-0.024317	0.058038	1.899427	0.014147	
S4_BM3	0.142890	1.812515	0.108966	
1.378075	0.112602	3.199181	0.033924	
S4_BM4	0.276255	3.037397	0.269690	
2.937032	0.021792	0.533145	0.006565	
S4_BM5	0.221058	1.841043	0.235950	
1.947108	-0.049429	-0.916357	-0.014892	
S5_BM1	0.019774	0.334903	-0.049362	
-0.872359	0.229480	9.110771	-0.029588	
S5_BM2	0.029546	0.512516	-0.031051	
-0.556116	0.201136	8.092622	-0.001505	
S5_BM3	0.091486	1.226788	0.053745	
0.720423	0.125272	3.772381	0.037741	
S5_BM4	0.020895	0.218806	-0.024922	
-0.260630	0.152078	3.572848	-0.004027	
S5_BM5	0.194488	1.473302	0.234433	
1.764546	-0.132586	-2.241941	-0.039945	

Sorted by absolute alpha reduction:

Portfolio	CAPM alpha	CAPM alpha t-stat	Market + RMW alpha	Market + RMW a
-----------	------------	-------------------	--------------------	----------------

lpha	t-stat	RMW	beta	RMW	beta	t-stat	Absolute	alpha	reduction
S1_BM1	-0.521850			-2.928735			-0.211694		
-1.331696	-1.029478			-14.548701			0.310156		
S2_BM1	-0.265817			-1.957654			-0.069699		
-0.551716	-0.650960			-11.575887			0.196118		
S3_BM1	-0.219664			-2.011970			-0.088994		
-0.852661	-0.433723			-9.335527			0.130670		
S4_BM1	-0.069075			-0.825770			0.020274		
0.250591	-0.296569			-8.235143			0.048802		
S5_BM3	0.091486			1.226788			0.053745		
0.720423	0.125272			3.772381			0.037741		
S4_BM3	0.142890			1.812515			0.108966		
1.378075	0.112602			3.199181			0.033924		
S4_BM2	0.015816			0.232109			-0.001669		
-0.024317	0.058038			1.899427			0.014147		
S4_BM4	0.276255			3.037397			0.269690		
2.937032	0.021792			0.533145			0.006565		
S3_BM3	0.143888			1.689005			0.139722		
1.624346	0.013829			0.361176			0.004166		
S5_BM2	0.029546			0.512516			-0.031051		
-0.556116	0.201136			8.092622			-0.001505		
S3_BM4	0.271832			2.848367			0.275619		
2.860227	-0.012571			-0.293062			-0.003787		
S5_BM4	0.020895			0.218806			-0.024922		
-0.260630	0.152078			3.572848			-0.004027		
S4_BM5	0.221058			1.841043			0.235950		
1.947108	-0.049429			-0.916357			-0.014892		
S3_BM5	0.343818			2.761454			0.358736		
2.854836	-0.049517			-0.885256			-0.014918		
S5_BM1	0.019774			0.334903			-0.049362		
-0.872359	0.229480			9.110771			-0.029588		
S3_BM2	0.124490			1.434342			0.157596		
1.807739	-0.109888			-2.831715			-0.033106		
S2_BM3	0.245106			2.367501			0.284879		
2.739681	-0.132014			-2.852144			-0.039773		
S5_BM5	0.194488			1.473302			0.234433		
1.764546	-0.132586			-2.241941			-0.039945		
S2_BM4	0.291446			2.746997			0.343806		
3.237785	-0.173793			-3.676875			-0.052360		
S2_BM5	0.319903			2.430130			0.405280		
3.096381	-0.283385			-4.863928			-0.085377		
S2_BM2	0.102585			0.917562			0.213308		
1.960328	-0.367516			-7.587664			-0.110723		
S1_BM5	0.473040			3.155047			0.598277		
4.055460	-0.415691			-6.330228			-0.125237		
S1_BM4	0.347082			2.644521			0.487168		
3.837622	-0.464978			-8.228621			-0.140086		
S1_BM3	0.137430			1.060841			0.296334		
2.399775	-0.527438			-9.595584			-0.158904		
S1_BM2	0.059483			0.378003			0.327770		
2.321462	-0.890504			-14.169017			-0.268286		

```
In [ ]: assert validation_passed, "Run the validation cell first."
```

```
T = len(common)
rmw_mean = common["RMW"].mean()
```

```

rmw_sd = common["RMW"].std(ddof=1)
rmw_se = rmw_sd / np.sqrt(T)

assert rmw_se > 0
rmw_t = rmw_mean / rmw_se

rmw_mean_test = pd.DataFrame([
    "T": T,
    "Mean monthly RMW (%)": rmw_mean,
    "Standard deviation (%)": rmw_sd,
    "Standard error (%)": rmw_se,
    "Mean t-statistic": rmw_t
])

print("One-sample test: H0: E[RMW] = 0")
print(rmw_mean_test.to_string(
    index=False, float_format=lambda x: f"{x:.6f}"
))
print("\nThis tests the RMW mean, not the joint portfolio pricing errors.")

```

One-sample test: H0: E[RMW] = 0

T	Mean monthly RMW (%)	Standard deviation (%)	Standard error (%)	Mean t-statistic
756	0.242804	2.250105	0.081836	2.966978

This tests the RMW mean, not the joint portfolio pricing errors.

```

In [12]: from scipy.stats import t as t_dist

return_sets = {
    "CAPM factors": F_capm,
    "Augmented factors": F_aug,
    "CAPM factors + portfolios": pd.concat([F_capm, Y], axis=1),
    "Augmented factors + portfolios": pd.concat([F_aug, Y], axis=1)
}

sr2 = {}

for name, returns in return_sets.items():
    mean = returns.mean().to_numpy()
    covariance = returns.cov().to_numpy()
    sr2[name] = float(mean @ np.linalg.solve(covariance, mean))

capm_gain = sr2["CAPM factors + portfolios"] - sr2["CAPM factors"]
aug_gain = sr2["Augmented factors + portfolios"] - sr2["Augmented factors"]
gain_reduction = (1 - aug_gain / capm_gain) * 100

print("SHARPE-RATIO RESULTS (monthly)")
for name, value in sr2.items():
    print(f"{name}: SR = {np.sqrt(value):.4f}, SR² = {value:.6f}")

print(f"\nCAPM incremental SR²: {capm_gain:.6f}")
print(f"Augmented incremental SR²: {aug_gain:.6f}")
print(f"Incremental SR² reduction: {gain_reduction:.6f}%")
print(f"CAPM scaling denominator: {1 + sr2['CAPM factors']:.4f}")

```

```

print(f"Augmented scaling denominator: {1 + sr2['Augmented factors']:.4f}")

rmw_p = 2 * t_dist.sf(abs(rmw_t), df=len(common) - 1)

print("\nFACTOR MEANS AND RMW MEAN TEST")
print(f"Mean RM-RF: {common['RM-RF'].mean():.6f}% per month")
print(f"Mean RMW: {rmw_mean:.6f}% per month")
print(f"RMW mean t-statistic: {rmw_t:.6f}")
print(f"RMW mean-test p-value: {rmw_p:.6f}")

capm_abs = capm_results["Alpha"].abs()
aug_abs = aug_results["Alpha"].abs()
alpha_reduction = capm_abs - aug_abs

print("\nABSOLUTE-ALPHA SUMMARY (monthly percentage points)")
print(f"CAPM total absolute alpha: {capm_abs.sum():.6f}")
print(f"Augmented total absolute alpha: {aug_abs.sum():.6f}")
print(f"Portfolios with lower absolute alpha: {(alpha_reduction > 0).sum()}")

```

SHARPE-RATIO RESULTS (monthly)

CAPM factors: SR = 0.1346, SR^2 = 0.018115

Augmented factors: SR = 0.1917, SR^2 = 0.036737

CAPM factors + portfolios: SR = 0.4044, SR^2 = 0.163506

Augmented factors + portfolios: SR = 0.4117, SR^2 = 0.169468

CAPM incremental SR^2 : 0.145390

Augmented incremental SR^2 : 0.132731

Incremental SR^2 reduction: 8.707282%

CAPM scaling denominator: 1.0181

Augmented scaling denominator: 1.0367

FACTOR MEANS AND RMW MEAN TEST

Mean RM-RF: 0.600794% per month

Mean RMW: 0.242804% per month

RMW mean t-statistic: 2.966978

RMW mean-test p-value: 0.003102

ABSOLUTE-ALPHA SUMMARY (monthly percentage points)

CAPM total absolute alpha: 4.948719

Augmented total absolute alpha: 5.288944

Portfolios with lower absolute alpha: 9 of 25

In []: **assert** validation_passed, "Run the validation cell first."

```

print("COMMON SAMPLE")
print("Start:", common.index.min())
print("End:", common.index.max())
print("T:", len(common))

print("\nGRS RESULTS")
print(
    f"CAPM: F = {capm_grs['GRS F-statistic']:.6f}, "
    f"p = {capm_grs['p-value']:.6e}"
)
print(
    f"Market + RMW: F = {aug_grs['GRS F-statistic']:.6f}, "

```

```

    f"p = {aug_grs['p-value']:.6e}"
)
print(f"GRS percentage reduction: {grs_reduction:.6f}%")

print("\nRMW MEAN TEST")
print(rmw_mean_test.to_string(
    index=False, float_format=lambda x: f"{x:.6f}"
))

print("\nMODEL COMPARISON")
print(comparison.to_string(
    index=False, na_rep="N/A",
    formatters={
        "GRS F-statistic": "{:.6f}".format,
        "p-value": "{:.6e}".format,
        "Average RMW return": "{:.6f}".format
    }
))

print("\nALPHA COMPARISON: ORIGINAL ORDER")
print(alpha_comparison.to_string(
    index=False, float_format=lambda x: f"{x:.6f}"
))

print("\nALPHA COMPARISON: SORTED COPY")
print(alpha_sorted.to_string(
    index=False, float_format=lambda x: f"{x:.6f}"
))

print("\nGRS VALIDATION")
print(validation_results.to_string(index=False))
print(grs_audit.to_string(index=False))

print("\nTECHNICAL NOTE")
print(
    "The implementation uses one common sample, workbook RM-RF and RF, "
    "and only RMW from the external CSV. RF is subtracted once from "
    "raw portfolio returns. GRS follows the assignment's displayed "
    "formula, using OLS residual covariance divided by T-K-1, factor "
    "covariance divided by T-1, and F(N, T-N-K) degrees of freedom."
)
print(
    "Special attention was required for the workbook's multiple header "
    "rows, YYYYMM dates, and the CSV's separate annual section. "
    "The portfolio labels preserve the original workbook ordering."
)
print(
    "Date alignment excludes 444 workbook months before July 1963 "
    "and two RMW months after June 2026 in these uploaded files. "
    "No missing observations remain in the 756-month common sample. "
)

```

COMMON SAMPLE

Start: 1963-07

End: 2026-06

T: 756

GRS RESULTS

CAPM: $F = 4.164336$, $p = 9.853287e-11$ Market + RMW: $F = 3.723390$, $p = 4.213838e-09$

GRS percentage reduction: 10.588612%

RMW MEAN TEST

T	Mean monthly RMW (%)	Standard deviation (%)	Standard error (%)	Mean t-statistic
756	0.242804	2.250105	0.081836	2.966978

MODEL COMPARISON

Model	Factors	T	N	K	GRS F-statistic	p-value	Average R
MW return							
CAPM	RM-RF	756	25	1	4.164336	9.853287e-11	
N/A							
Augmented model	RM-RF + RMW	756	25	2	3.723390	4.213838e-09	
0.242804							

ALPHA COMPARISON: ORIGINAL ORDER

Portfolio	CAPM alpha	CAPM alpha t-stat	Market + RMW alpha	Market + RMW alpha t-stat	RMW beta	RMW beta t-stat	Absolute alpha reduction
S1_BM1	-0.521850	-2.928735	-0.211694	-1.331696	-1.029478	-14.548701	0.310156
S1_BM2	0.059483	0.378003	0.327770	2.321462	-0.890504	-14.169017	-0.268286
S1_BM3	0.137430	1.060841	0.296334	2.399775	-0.527438	-9.595584	-0.158904
S1_BM4	0.347082	2.644521	0.487168	3.837622	-0.464978	-8.228621	-0.140086
S1_BM5	0.473040	3.155047	0.598277	4.055460	-0.415691	-6.330228	-0.125237
S2_BM1	-0.265817	-1.957654	-0.069699	-0.551716	-0.650960	-11.575887	0.196118
S2_BM2	0.102585	0.917562	0.213308	1.960328	-0.367516	-7.587664	-0.110723
S2_BM3	0.245106	2.367501	0.284879	2.739681	-0.132014	-2.852144	-0.039773
S2_BM4	0.291446	2.746997	0.343806	3.237785	-0.173793	-3.676875	-0.052360
S2_BM5	0.319903	2.430130	0.405280	3.096381	-0.283385	-4.863928	-0.085377
S3_BM1	-0.219664	-2.011970	-0.088994	-0.852661	-0.433723	-9.335527	0.130670
S3_BM2	0.124490	1.434342	0.157596	1.807739	-0.109888	-2.831715	-0.033106
S3_BM3	0.143888	1.689005	0.139722	1.624346	0.013829	0.361176	0.004166
S3_BM4	0.271832	2.848367	0.275619	2.860227	-0.012571	-0.293062	-0.003787
S3_BM5	0.343818	2.761454	0.358736				

2.854836	-0.049517	-0.885256	-0.014918
S4_BM1	-0.069075	-0.825770	0.020274
0.250591	-0.296569	-8.235143	0.048802
S4_BM2	0.015816	0.232109	-0.001669
-0.024317	0.058038	1.899427	0.014147
S4_BM3	0.142890	1.812515	0.108966
1.378075	0.112602	3.199181	0.033924
S4_BM4	0.276255	3.037397	0.269690
2.937032	0.021792	0.533145	0.006565
S4_BM5	0.221058	1.841043	0.235950
1.947108	-0.049429	-0.916357	-0.014892
S5_BM1	0.019774	0.334903	-0.049362
-0.872359	0.229480	9.110771	-0.029588
S5_BM2	0.029546	0.512516	-0.031051
-0.556116	0.201136	8.092622	-0.001505
S5_BM3	0.091486	1.226788	0.053745
0.720423	0.125272	3.772381	0.037741
S5_BM4	0.020895	0.218806	-0.024922
-0.260630	0.152078	3.572848	-0.004027
S5_BM5	0.194488	1.473302	0.234433
1.764546	-0.132586	-2.241941	-0.039945

ALPHA COMPARISON: SORTED COPY

Portfolio	CAPM alpha	CAPM alpha t-stat	Market + RMW alpha	Market + RMW alpha t-stat	Market + RMW alpha reduction
S1_BM1	-0.521850	-2.928735	-0.211694		
-1.331696	-1.029478	-14.548701	0.310156		
S2_BM1	-0.265817	-1.957654	-0.069699		
-0.551716	-0.650960	-11.575887	0.196118		
S3_BM1	-0.219664	-2.011970	-0.088994		
-0.852661	-0.433723	-9.335527	0.130670		
S4_BM1	-0.069075	-0.825770	0.020274		
0.250591	-0.296569	-8.235143	0.048802		
S5_BM3	0.091486	1.226788	0.053745		
0.720423	0.125272	3.772381	0.037741		
S4_BM3	0.142890	1.812515	0.108966		
1.378075	0.112602	3.199181	0.033924		
S4_BM2	0.015816	0.232109	-0.001669		
-0.024317	0.058038	1.899427	0.014147		
S4_BM4	0.276255	3.037397	0.269690		
2.937032	0.021792	0.533145	0.006565		
S3_BM3	0.143888	1.689005	0.139722		
1.624346	0.013829	0.361176	0.004166		
S5_BM2	0.029546	0.512516	-0.031051		
-0.556116	0.201136	8.092622	-0.001505		
S3_BM4	0.271832	2.848367	0.275619		
2.860227	-0.012571	-0.293062	-0.003787		
S5_BM4	0.020895	0.218806	-0.024922		
-0.260630	0.152078	3.572848	-0.004027		
S4_BM5	0.221058	1.841043	0.235950		
1.947108	-0.049429	-0.916357	-0.014892		
S3_BM5	0.343818	2.761454	0.358736		
2.854836	-0.049517	-0.885256	-0.014918		
S5_BM1	0.019774	0.334903	-0.049362		
-0.872359	0.229480	9.110771	-0.029588		
S3_BM2	0.124490	1.434342	0.157596		

1.807739	-0.109888	-2.831715	-0.033106
S2_BM3	0.245106	2.367501	0.284879
2.739681	-0.132014	-2.852144	-0.039773
S5_BM5	0.194488	1.473302	0.234433
1.764546	-0.132586	-2.241941	-0.039945
S2_BM4	0.291446	2.746997	0.343806
3.237785	-0.173793	-3.676875	-0.052360
S2_BM5	0.319903	2.430130	0.405280
3.096381	-0.283385	-4.863928	-0.085377
S2_BM2	0.102585	0.917562	0.213308
1.960328	-0.367516	-7.587664	-0.110723
S1_BM5	0.473040	3.155047	0.598277
4.055460	-0.415691	-6.330228	-0.125237
S1_BM4	0.347082	2.644521	0.487168
3.837622	-0.464978	-8.228621	-0.140086
S1_BM3	0.137430	1.060841	0.296334
2.399775	-0.527438	-9.595584	-0.158904
S1_BM2	0.059483	0.378003	0.327770
2.321462	-0.890504	-14.169017	-0.268286

GRS VALIDATION

Model	Check	GRS Result
CAPM	Zero alpha	0.000000 PASS
CAPM Reorder	invariance	4.164336 PASS
CAPM Scale	invariance	4.164336 PASS
Market + RMW	Zero alpha	0.000000 PASS
Market + RMW Reorder	invariance	3.723390 PASS
Market + RMW Scale	invariance	3.723390 PASS

Model	K	Residual denominator	Factor denominator	F	degrees of freedom
om OLS residual check Orthogonality Specification					
CAPM	1	754	755		(25, 73)
0)	PASS	PASS	PASS		
Market + RMW	2	753	755		(25, 72)
9)	PASS	PASS	PASS		

TECHNICAL NOTE

The implementation uses one common sample, workbook RM-RF and RF, and only R MW from the external CSV. RF is subtracted once from raw portfolio returns. GRS follows the assignment's displayed formula, using OLS residual covariance divided by $T-K-1$, factor covariance divided by $T-1$, and $F(N, T-N-K)$ degrees of freedom.

Special attention was required for the workbook's multiple header rows, YYYY MM dates, and the CSV's separate annual section. The portfolio labels preserve the original workbook ordering.

Date alignment excludes 444 workbook months before July 1963 and two RMW months after June 2026 in these uploaded files. No missing observations remain in the 756-month common sample. Record any actual local changes or errors separately in your AI log.

```
In [14]: pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", None)
pd.set_option("display.max_colwidth", None)

print(alpha_comparison.to_string(
    index=False,
```

```
float_format=lambda x: f"{x:.6f}"
))
```

Portfolio	CAPM alpha lpha t-stat	CAPM alpha t-stat RMW beta t-stat	Market + RMW alpha Absolute alpha reduction
S1_BM1	-0.521850	-2.928735	-0.211694
-1.331696	-1.029478	-14.548701	0.310156
S1_BM2	0.059483	0.378003	0.327770
2.321462	-0.890504	-14.169017	-0.268286
S1_BM3	0.137430	1.060841	0.296334
2.399775	-0.527438	-9.595584	-0.158904
S1_BM4	0.347082	2.644521	0.487168
3.837622	-0.464978	-8.228621	-0.140086
S1_BM5	0.473040	3.155047	0.598277
4.055460	-0.415691	-6.330228	-0.125237
S2_BM1	-0.265817	-1.957654	-0.069699
-0.551716	-0.650960	-11.575887	0.196118
S2_BM2	0.102585	0.917562	0.213308
1.960328	-0.367516	-7.587664	-0.110723
S2_BM3	0.245106	2.367501	0.284879
2.739681	-0.132014	-2.852144	-0.039773
S2_BM4	0.291446	2.746997	0.343806
3.237785	-0.173793	-3.676875	-0.052360
S2_BM5	0.319903	2.430130	0.405280
3.096381	-0.283385	-4.863928	-0.085377
S3_BM1	-0.219664	-2.011970	-0.088994
-0.852661	-0.433723	-9.335527	0.130670
S3_BM2	0.124490	1.434342	0.157596
1.807739	-0.109888	-2.831715	-0.033106
S3_BM3	0.143888	1.689005	0.139722
1.624346	0.013829	0.361176	0.004166
S3_BM4	0.271832	2.848367	0.275619
2.860227	-0.012571	-0.293062	-0.003787
S3_BM5	0.343818	2.761454	0.358736
2.854836	-0.049517	-0.885256	-0.014918
S4_BM1	-0.069075	-0.825770	0.020274
0.250591	-0.296569	-8.235143	0.048802
S4_BM2	0.015816	0.232109	-0.001669
-0.024317	0.058038	1.899427	0.014147
S4_BM3	0.142890	1.812515	0.108966
1.378075	0.112602	3.199181	0.033924
S4_BM4	0.276255	3.037397	0.269690
2.937032	0.021792	0.533145	0.006565
S4_BM5	0.221058	1.841043	0.235950
1.947108	-0.049429	-0.916357	-0.014892
S5_BM1	0.019774	0.334903	-0.049362
-0.872359	0.229480	9.110771	-0.029588
S5_BM2	0.029546	0.512516	-0.031051
-0.556116	0.201136	8.092622	-0.001505
S5_BM3	0.091486	1.226788	0.053745
0.720423	0.125272	3.772381	0.037741
S5_BM4	0.020895	0.218806	-0.024922
-0.260630	0.152078	3.572848	-0.004027
S5_BM5	0.194488	1.473302	0.234433
1.764546	-0.132586	-2.241941	-0.039945

3a report

What the code got right.

The code correctly constructed portfolio excess returns, used the same 1963-07 to 2026-06 common sample for both models, and estimated the CAPM with ($K=1$) and the augmented model with ($K=2$). The GRS implementation used the residual covariance matrix with denominator $(T-K-1)$, the factor covariance matrix with denominator $(T-1)$, the correct scaling term, and $(F(N,T-N-K))$ degrees of freedom. The zero-alpha, reorder-invariance, and scale-invariance checks also passed, and the independently recomputed GRS statistics matched the notebook.

What I corrected or added.

I did not need to correct the regression or GRS calculations themselves. The main correction was to the alpha-table display: the original notebook output was visually truncated, so I added a separate cell that prints all 25 portfolios for Step 3(b). Additional Sharpe-ratio and summary statistics were added later for Step 4, rather than to fix an error in the Step 3 implementation.

Step 3b

LLM's Explanation: Adding RMW helps price a relatively small subset of portfolios, especially the low-BE/ME portfolios. The largest improvements are for S1_BM1 (absolute alpha falls by 0.310), S2_BM1 (0.196), and S3_BM1 (0.131). Smaller improvements occur for portfolios such as S4_BM1, S4_BM3, and S5_BM3.

RMW does not help many of the remaining portfolios. The largest deteriorations are S1_BM2 (-0.268), S1_BM3 (-0.159), S1_BM4 (-0.140), and S1_BM5 (-0.125). Overall, RMW appears most useful for some low-BE/ME portfolios, particularly among smaller firms, but it does not consistently reduce pricing errors across the full set of 25 portfolios.

Critical Assessing:

The LLM's portfolio-level figures are accurate, and it correctly identifies the largest reductions (S1_BM1, S2_BM1, S3_BM1) and deteriorations (S1_BM2-S1_BM5) in absolute alpha. However, it understates how limited the improvement is: absolute alphas fall for only 9 of 25 portfolios, several of those reductions are below 0.015, and total absolute alpha rises from 4.95 to 5.29. Its claim that RMW helps "particularly among smaller firms" is misleading, because the smallest-size quintile also contains the four largest deteriorations. Most importantly, it misses the pattern that organizes the table: given RMW's positive sample mean, pricing errors shrink only where the sign of the RMW loading offsets the sign of the CAPM alpha, and grow where the loading pushes an

already-positive alpha further from zero, as in small value portfolios. The table therefore only partly supports my hypothesis: weakly exposed portfolios change little, but strong RMW exposure is associated with large alpha changes in either direction, not necessarily reductions.

Step 4

Model	Factors	GRS F-statistic	p-value	Mean RM-RF	Mean RMW
CAPM	RM-RF	4.164336	9.853287e-11	0.600794%	—
Augmented	RM-RF + RMW	3.723390	4.213838e-09	0.600794%	0.242804%

4a)

Adding RMW lowers GRS from 4.164336 to 3.723390, a 10.588612% reduction. This indicates improvement in the joint pricing-error criterion, but the reduction itself is not a formal test of statistically significant improvement. The augmented-model p-value remains 4.213838e-09, strongly rejecting the joint null that all 25 alphas equal zero.

Using the same uploaded sample and sample covariances with denominator $T - 1$, the incremental maximum squared monthly Sharpe ratio from adding the test assets falls from 0.145390 to 0.132731, approximately 8.707282%. Thus, the test assets offer a smaller remaining Sharpe-ratio improvement, but no combination of RM-RF and RMW is mean-variance efficient relative to the factors plus the 25 test assets. Most of the change comes from the factor side: the maximum monthly Sharpe ratio of the factors rises from 0.1346 (market alone) to 0.1917 (market + RMW), while that of the factors plus test assets rises only from 0.4044 to 0.4117. The percentage differs from the GRS reduction mainly because GRS divides this increment by $1 + SR_f^2$, which rises from 1.0181 to 1.0367 when RMW is added; the remainder reflects the degrees-of-freedom and covariance-denominator adjustments.

4b)

Defining improvement as $|\alpha_{\text{CAPM}}| - |\alpha_{\text{augmented}}|$, the largest reductions occur in S1_BM1 (0.310156), S2_BM1 (0.196118), and S3_BM1 (0.130670), measured in monthly percentage points. These are the lowest-BE/ME portfolios in the first three size quintiles.

However, absolute alphas decline for only 9 of 25 portfolios. The largest deteriorations occur in S1_BM2 (−0.268286), S1_BM3 (−0.158904), S1_BM4 (−0.140086), and S1_BM5 (−0.125237). Stronger absolute RMW exposure therefore does not consistently imply improvement: S1_BM2 has a large loading of −0.890504, yet its positive alpha increases substantially.

The hypothesis is partially supported. GRS falls and pricing errors remain, as predicted, and the three largest reductions occur in portfolios with strong negative RMW loadings ($-1.029, -0.651, -0.434$). Weakly exposed portfolios ($|\beta_{RMW}| < 0.06$) change by at most 0.015, also consistent with the hypothesis. However, strong exposure is associated with large alpha changes in either direction rather than consistently larger reductions: of the seven portfolios with $|\beta_{RMW}| \geq 0.4$, three improve and four deteriorate.

4c)

1. RMW's mean return: The average is 0.242804% per month, with $t = 2.966978$. A conventional two-sided t-test rejects a zero mean at the 1% level ($p \approx 0.003102$).
2. Pricing-error improvement: GRS decreases by 10.588612%, indicating a modest improvement in the joint criterion. Nevertheless, the augmented-model p-value of 4.213838e-09 and the mixed portfolio-level alpha changes show that substantial pricing errors remain.

These tests address different questions. The mean-return test asks whether RMW's average return, its implied premium in the time-series model, differs from zero; GRS tests whether the 25 intercepts are jointly zero, accounting for residual covariances and factor moments. A factor with an imprecisely estimated mean can still reduce pricing errors, because each portfolio's alpha change depends on its RMW loading and the point estimate of the mean, not on the precision of that estimate. Conversely, RMW's significant mean here does not imply that it adequately prices the test assets.

4d)

The main limitation is overlap with the evidence used to establish RMW. Fama and French (2015) documented the five-factor model over July 1963–December 2013 and evaluated it on the same 25 size–B/M portfolios, so about 80% of this test's 756 months (606), and the test assets themselves, overlap the factor's discovery sample; only January 2014–June 2026 (150 months) postdates it. Factor construction adds further dependence: RMW and the test portfolios are formed from the same U.S. CRSP/Compustat universe using NYSE-based breakpoints, so they share underlying stocks. This is therefore not an independent out-of-sample test, and the historical GRS reduction may overstate performance in new data; this limits external validity without invalidating the in-sample result.

Prompt 1 — Step 3(a): Implementation Write Python code to estimate the CAPM and augmented RM-RF + RMW time-series regressions for the 25 Size/BE/ME portfolios and run the GRS test. Use the same common sample for both models, report the GRS statistics and p-values, create an alpha comparison table, and validate the GRS implementation.

Prompt 2 — Step 3(b): Interpretation Using the 25-portfolio alpha comparison table, identify which portfolios RMW helps price and which it does not. Focus on changes in

absolute alpha, patterns across size and BE/ME, and whether the results are consistent with my Step 2 hypothesis.

Step 5

AI interaction Log

Interaction	Prompt/Purpose	What the LLM produced correctly	What required correction or verification
Step 3(a): Implementation	Prompt 1 above: implement and validate CAPM and RM-RF + RMW regressions and GRS tests.	The final implementation correctly handled the two-factor regression and the assignment's GRS framework, producing GRS statistics, p-values, and portfolio alphas.	I verified the common sample, excess-return construction, K=1 versus K=2, covariance denominators, degrees of freedom, and zero-alpha, reorder-invariance, and scale-invariance checks. No calculation correction was needed; I expanded the truncated alpha-table display to show all 25 portfolios and added a cell computing the Sharpe-ratio quantities, RMW mean-test p-value, mean RM-RF, and absolute-alpha totals.
Step 3(b): Alpha interpretation	Prompt 2 above: interpret absolute-alpha changes, size/BE/ME patterns, and consistency with my hypothesis.	All quoted figures were accurate, including the largest improvements in S1_BM1–S3_BM1 and deteriorations in S1_BM2–S1_BM5.	The description needed precision: only 9 of 25 portfolios improved, and "particularly among smaller firms" obscured the four largest deteriorations within the smallest-size quintile. The response omitted the hypothesis comparison and the role of signed RMW exposures in explaining alpha changes; significant exposure alone would not establish improvement.

Working with the LLM clarified that GRS jointly evaluates regression intercepts using the residual covariance matrix, accounting for correlations among pricing errors. Because GRS evaluates the alphas using the inverse residual covariance matrix, its statistic can fall even when most individual absolute alphas rise, as in my results. Verifying covariance denominators and degrees of freedom showed me why invariance checks alone cannot establish that an implementation is correct. I now understand GRS as a test of the factors' mean-variance efficiency relative to the test assets, rather than a count of individually significant alphas.